

VERIFICATION

Semgrep Integration — Verification Protocol

Field	Value
Revision	1
Last modified	2026-06-21T00:00:00Z
Status	active
Cross-references	scripts/semgrep/semgrep_ci_test.sh, scripts/hooks/semgrep_precommit.sh, .docs_chain/contexts/semgrep_status.yaml, docs/codegraph/Status_Summary.md

1. Purpose

This document defines the verification protocol for Semgrep SAST (Static Application Security Testing) integration in this project. Every verification step **MUST** produce captured evidence per §11.4.69 and **MUST** be reproducible by any agent or operator.

2. Installation Verification

2.1 Semgrep Installed

`which semgrep`

- **Expected:** prints a path (e.g., `/usr/local/bin/semgrep`)
- **Evidence:** `qa-results/semgrep_ci_<ts>/tmp/version.txt`
- **Fail:** `which semgrep` returns non-zero → install via `pip install semgrep`

2.2 Semgrep Version

```
semgrep --version
```

- **Expected:** prints version string (e.g., 1.83.0)
- **Evidence:** qa-results/<run-id>/tmp/version.txt
- **Fail:** version check returns non-zero → reinstall or check PATH

2.3 Registry Connectivity

```
semgrep --config auto --dry-run /dev/null
```

- **Expected:** exits 0, downloads or caches registry rules
- **Evidence:** qa-results/<run-id>/tmp/registry_check.txt
- **Fail:** network error → verify internet connectivity or configure proxy

3. CI Test Verification

3.1 Run the CI Test Script

```
bash scripts/semgrep/semgrep_ci_test.sh
```

- **Expected:** exit 0, all checks PASS
- **Evidence:** qa-results/semgrep_ci_<ts>/evidence.log + summary.txt
- **Fail:** see evidence.log for per-check failure details

3.2 CI Test Checks (auto-verified by the script)

Check	Description	Expected
PATH check	which semgrep succeeds	PASS
Version check	semgrep --version returns 0	PASS
Vuln de-tection	semgrep scan --config auto on Python eval fixture	PASS (exit 2, ≥1 results)
Shell scan	semgrep scan --config auto on .sh files	PASS (exit 0 or 2, no crash)

4. Pre-Commit Hook Verification

4.1 Hook Script Exists

```
ls -la scripts/hooks/semgrep_precommit.sh
```

- **Expected:** file exists, is executable
- **Evidence:** `ls -la` output
- **Fail:** file missing → copy from constitution/scripts/hooks/

4.2 Hook Syntax Valid

```
sh -n scripts/hooks/semgrep_precommit.sh
```

- **Expected:** exit 0, no output
- **Fail:** syntax error → fix before using

4.3 Hook Wired in .git/hooks/

```
ls -la .git/hooks/pre-commit
grep -q semgrep .git/hooks/pre-commit 2>/dev/null && echo
      "PRESENT" || echo "NOT WIRED"
```

- **Expected:** "PRESENT"
- **Fail:** run `scripts/install_git_hooks.sh` or symlink manually:

```
ln -sf ../../scripts/hooks/semgrep_precommit.sh .git/hooks/pre-commit
```

4.4 Hook Functional Test

Create a staged Python file with a known vulnerability:

```
echo 'x = eval(input())' > /tmp/semgrep_test_vuln.py
git add /tmp/semgrep_test_vuln.py 2>/dev/null
```

Run `scripts/hooks/semgrep_precommit.sh`:

- **Expected:** exit 1, prints "COMMIT BLOCKED"
- **Clean up:** `rm -f /tmp/semgrep_test_vuln.py`

Create a staged clean Python file:

```
echo 'x = 42' > /tmp/semgrep_test_clean.py
git add /tmp/semgrep_test_clean.py 2>/dev/null
```

Run `scripts/hooks/semgrep_precommit.sh`:

- **Expected:** exit 0, prints "no issues found"

5. MCP Agent Integration Verification

5.1 Semgrep MCP Plugin Installed

Verify the Semgrep MCP is available in the agent's capability list.

For Claude Code:

```
grep -r semgrep .claude/settings.json 2>/dev/null || echo "CHECK  
MANUAL CONFIG"
```

- **Expected:** semgrep MCP server is configured
- **Evidence:** .claude/settings.json snippet with semgrep entry

5.2 MCP Tool Functional Check

Invoke the semgrep MCP tool against a test fixture:

Execute `semgrep_scan` with `code_files` containing a Python `eval()` fixture

- **Expected:** returns ≥ 1 finding
- **Fail:** MCP tool returns error — check semgrep server logs

6. Anti-Bluff Evidence Protocol (§11.4.69)

Every verification step MUST produce a captured-evidence artefact:

6.1 Evidence Storage Layout

```
qa-results/  
  semgrep_ci_<ts>/  
    evidence.log          # Chronological event log with PASS/  
FAIL/SKIP  
  summary.txt            # One-line summary for automation  
  tmp/  
    version.txt          # semgrep --version output  
    vuln_test.py         # Test fixture (eval injection)  
    scan_results.json    # JSON results from vulnerability  
scan  
  sh_scan_results.json   # JSON results from shell-script  
scan
```

6.2 Evidence Assertions

Artefact	Required	Validates
version.txt	Non-empty, semgrep version string	Installation
scan_results.json	Contains ≥ 1 "check_id" key	Vulnerability detection
sh_scan_results.json	Contains valid JSON (0 or more findings)	Shell-script scan
evidence.log	Contains PASS count	All checks executed

6.3 Self-Validation (Golden Fixtures)

The `semgrep_ci_test.sh` script ships with:

- **Golden-bad fixture:** `vuln_test.py` containing `eval(input())` — MUST produce ≥ 1 semgrep finding
- **Golden-good pattern:** clean Python code — MUST produce 0 findings

Replace the test fixture with a clean file and re-run:

```
echo 'x = 42' > "${SEMGREP_TMP}/vuln_test.py"
bash scripts/semgrep/semgrep_ci_test.sh
```

- **Expected:** vulnerability-detection check FAILs (the fixture is now clean)
- **Restore:** re-run the real script (regenerates the fixture)

This proves the gate is NOT a bluff — it genuinely catches violations.

7. Troubleshooting

9. Build-From-Source via Git Submodule

The semgrep source repository is registered as a git submodule at `submodules/semgrep` (OCaml project). Building from source is **optional** — the recommended install path remains `pip install --user semgrep` (Section 2). The submodule exists for reference, local inspection, and for operators who wish to build semgrep with the OCaml toolchain enabled.

9.1 Initialize the Submodule

```
git submodule update --init submodules/semgrep
```

9.2 Build from Source

```
# Source the setup script for the semgrep_setup_build function
. scripts/semgrep/semgrep_setup.sh
semgrep_setup_build
```

Prerequisites: ocaml, opam, make, gcc.

If the OCaml toolchain is unavailable, semgrep_setup_build exits with code 2 and advises the pip path.

9.3 Submodule Layout

```
constitution/
  submodules/semgrep/      # semgrep OCaml source (git
submodule)                 # submodule)
  scripts/semgrep/         # Setup, validation, CI scripts
  docs/semgrep/            # Documentation
```

7.1 Semgrep Not Found

```
pip install semgrep
# or: pip3 install semgrep
# Verify: semgrep --version
```

7.2 No Rules Firing on Test Fixture

```
# Test with explicit rule
semgrep scan --config auto --patterns '["eval(...)"]' testfile.py
# Check registry connectivity
semgrep --config auto --dry-run /dev/null
```

7.3 Pre-Commit Hook Not Firing

```
# Verify hook is installed
ls -la .git/hooks/pre-commit
# Verify hook calls semgrep
grep semgrep .git/hooks/pre-commit || echo "HOOK NOT WIRED"
```

7.4 MCP Integration Not Working

```
# Verify semgrep MCP is registered in agent settings
grep -r "semgrep" ~/.claude/ 2>/dev/null
```

```
# Try direct semgrep scan from command line
semgrep scan --config auto --json /path/to/testfile.py
```

8. Cross-References

File	Purpose
submodules/semgrep	semgrep OCaml source (git sub-module, optional build-from-source)
scripts/semgrep/ semgrep_ci_test.sh	CI-ready validation test
scripts/hooks/ semgrep_precommit.sh	Pre-commit hook
.docs_chain/contexts/ semgrep_status.yaml	docs_chain auto-sync context
docs/codegraph/ Status_Summary.md	Integration status summary